

2026.02.23.월

🕒 생성일	@2026년 2월 23일 오전 8:32
🏷 태그	Docker

목차

1. Docker Compose
2. Docker Swarm
3. Docker Network

▼ 1) Docker Compose

- 정의
 - 여러 개의 컨테이너를 하나의 서비스 단위로 정의·실행·관리하는 도구
- 기능
 - 프로덕션, 스테이징, 개발, 테스트 및 CI 워크플로
- 프로세스
 - 환경 정의
 - docker-compose.yml 작성
 - docker compose up 으로 명령 실행
- 기본 명령어

```
# 도커 컴포즈 실행
docker compose up -d
# 캐시 지우고 이미지 재빌딩
docker compose up -d --build

# 도커 컴포즈에 정의된 서비스 컨테이너 정지, 삭제
docker compose down

# 도커 컴포즈에 정의된 서비스 정지, 시작
docker compose stop
docker compose start

# 도커 컴포즈에 정의된 서비스 목록
docker compose ps
```

▼ 2) Docker Swarm

- 도커 오케스트레이션(클러스터링) 도구
 - 쿠버네티스
 - AWS ECS
 - Apache Mesos
 - Docker swarm
- 주요 기능
 - 프로비저닝 및 배포
 - 구성 및 일정 조정
 - 리소스 할당
 - 컨테이너 가용성 관리

- 컨테이너 스케일링 또는 제거
- 로드 밸런싱 및 트래픽 라우팅
- 컨테이너 모니터링
- 실행된 컨테이너 기반 애플리케이션 설정
- 컨테이너 간 보안 유지
- 정의
 - 서로 다른 호스트(서버)에 있는 여러 대의 컨테이너를 묶어 하나의 호스트인 것처럼 사용할 수 있게 해주는 도구
- 핵심 구조
 - 분산 코디네이터: 여러 도커 서버를 클러스터로 구성하기 위한 정보를 저장 및 동기화
 - 에이전트: 각 서버를 제어
 - 워커 노드: 실제 컨테이너가 생성되고 관리되는 도커 서버, 0개 이상 존재
 - 매니저 노드: 워커 노드를 관리, 반드시 1개 이상 존재
- 뗏목 합의 알고리즘 (멀티 매니저 노드)
 - 매니저 노드가 중단되면 클러스터 전체가 다운되는 문제 발생
 - 여러 개의 매니저 노드를 생성하여 문제 발생시 서비스 복원
 - 모든 매니저 노드가 동일한 상태를 유지해야 함.
- 합의 알고리즘
 - 합의 요청: 한 서버가 명령 실행을 위해 다른 서버에게 합의 요청
 - 요청 받은 서버들은 요청한 서버의 상태 체크
 - 동기화: 합의한 서버는 같은 명령을 실행하고 동일한 상태 유지
 - 로그로부터 입력 받으며, 오류를 허용하는 특징을 가짐
 - 중복 방지: 이미 합의한 서버에게는 다시 합의를 구할 필요 없음
 - 오류 처리: 오류가 발생한 서버가 과반수라면 명령 진행 중단
 - 매니저 노드는 가능한 홀수 개로 구성
- 장애 허용 분산 시스템
 - 시스템 중 일부에 오류가 발생해도 정상적으로 작동할 수 있는 시스템
- docker swarm 실습

```
# ec2에 도커 설치 이후 (마스터 노드)
# AMI 이미지로 새로운 ec2 두 개 추가 생성(워커 노드*2)
# 마스터 ec2에 도커 파일, 도커 컴포즈, 이미지 빌드 완료한 상태

# 각 인스턴스 별로 호스트명 변경
sudo hostnamectl set-hostname <원하는 호스트명>

# 스웜 활성화 및 마스터 노드 지정
docker swarm init

# 스웜이 활성화 되었는지 확인(swarm: active)
docker info

# 마스터 노드에 워커 노드 추가(매니저는 worker 대신 manager)
docker swarm join-token worker

# 워커 서버에 생성된 명령어 복붙
docker swarm join --token <토큰 값> <manger-private-ip>:<port>

# 마스터 노드에서 노드 목록 확인
docker node ls
```

```
# 도커허브 로그인
docker login

# 도커 태그 설정
# 이때 새 이미지명 형식은 '유저명/이미지명'
docker tag <로컬 이미지명> <새 이미지명>

# 도커 허브에 이미지 푸시
docker push <유저명>/<이미지명>

# 도커 스웸 서비스 생성 (이미지명 = 도커 허브에 올라간 이미지 '유저명/이미지명', 도커 허브의 공식이미지는 유저명 생략 가능)
docker service create --name <사용할 서비스명> -p <포트>:<포트> <사용할 이미지명>
# 글로벌 서비스 생성
docker service create --name <사용할 서비스명> -p <포트>:<포트> --mode global <사용할 이미지명>

# 서비스 작동 상태 확인
docker service ps <서비스명>

# 전체 서비스 목록 확인
docker service ls

# 스케일링(로드 밸런싱 및 가용성 확보)
docker service scale <서비스명>=<할당할 노드 수>

# 서비스 삭제
docker service rm <서비스명>
```

- 서비스 모드
 - replicated
 - 지정한 개수만큼 컨테이너 실행 (`--replicas 3` 이면 3개)
 - 부하에 따라 컨테이너 수 조절 가능. 웹서버, API 등 스케일링 필요한 서비스에 적합.
 - global
 - 모든 노드에 컨테이너 1개씩 자동 실행, 노드 추가되면 자동 배포
 - **모든 노드에 반드시 1개 보장.** 모니터링 에이전트, 로그 수집기 등 노드마다 꼭 있어야 하는 서비스에 적합.

▼ 3) Docker Network

- 네트워크 드라이버 목록

```
docker network ls
```

드라이버	설명	사용 시점
bridge	단일 호스트 내부 네트워크 (기본값)	가장 일반적인 경우
host	호스트 네트워크 직접 사용	고성능, 포트 매핑 불필요
none	네트워크 완전 차단	보안 테스트
overlay	여러 노드 간 네트워크 (Swarm)	클러스터 환경
macvlan	물리 네트워크에 직접 IP 부여	L2 네트워크 통합 필요 시

- 사용자 정의 bridge 생성

```
docker network create my-network
```

- 컨테이너 실행시 네트워크 연결

```
docker run -d --name app --network my-network nginx
```

- bridge 네트워크 특징
 - 같은 네트워크에 있는 컨테이너는 **이름으로 통신 가능**
 - 자동 DNS 제공
 - 네트워크 격리 가능
- host 네트워크 특징
 - 포트 매핑 불필요 (-p 필요 없음)
 - 컨테이너가 호스트 IP 그대로 사용
 - 네트워크 격리 없음
 - 성능 ↑, 보안 ↓
 - Linux에서만 정상 지원.
- 네트워크 기본 명령어

명령어	설명
docker network ls	네트워크 목록
docker network inspect	상세 정보
docker network create	네트워크 생성
docker network rm	삭제
docker network connect	컨테이너 추가 연결
docker network disconnect	연결 해제

- Docker Network 기본 구조

구성요소	역할
network namespace	컨테이너 독립 네트워크 공간
veth pair	가상 케이블 (컨테이너 ↔ 브리지 연결)
docker0	기본 bridge 네트워크
iptables	NAT 및 포트 포워딩
Host NIC	실제 물리 네트워크 인터페이스

- 내부 동작
 1. network namespace 생성
 2. veth pair 생성
 3. 한쪽은 컨테이너 eth0에 연결
 4. 다른 한쪽은 docker0 bridge에 연결
 5. 컨테이너에 내부 IP 할당 (예: 172.17.0.2)
 6. iptables에 NAT 룰 추가